



Technische
Universität
Dresden



Distributed Computing

Digitization and Data Analytics

Structure of the Lecture Series

Lecture 1: Introduction

Lecture 2: Data Preprocessing

Lecture 3: Research Data Management

Lecture 4: Data Analytics and Statistics

Lecture 5: Machine Learning I

Lecture 6: Machine Learning II

Lecture 7: Machine Learning III

Lecture 8: Deep Learning I

Lecture 9: Deep Learning II

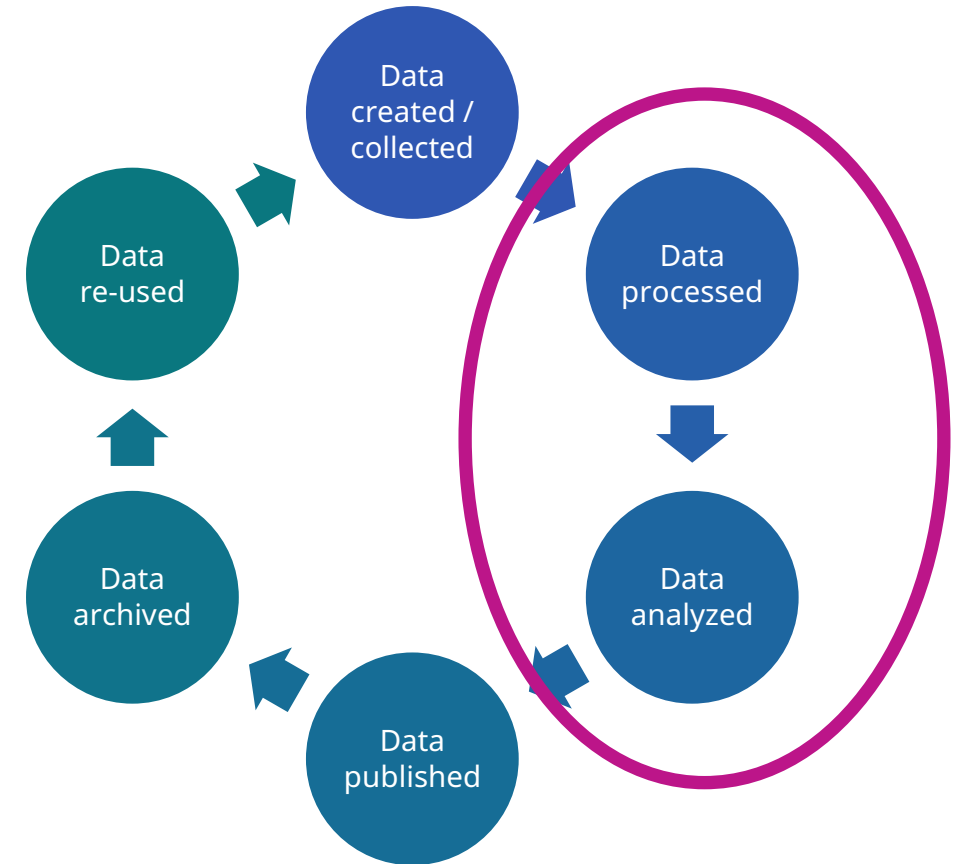
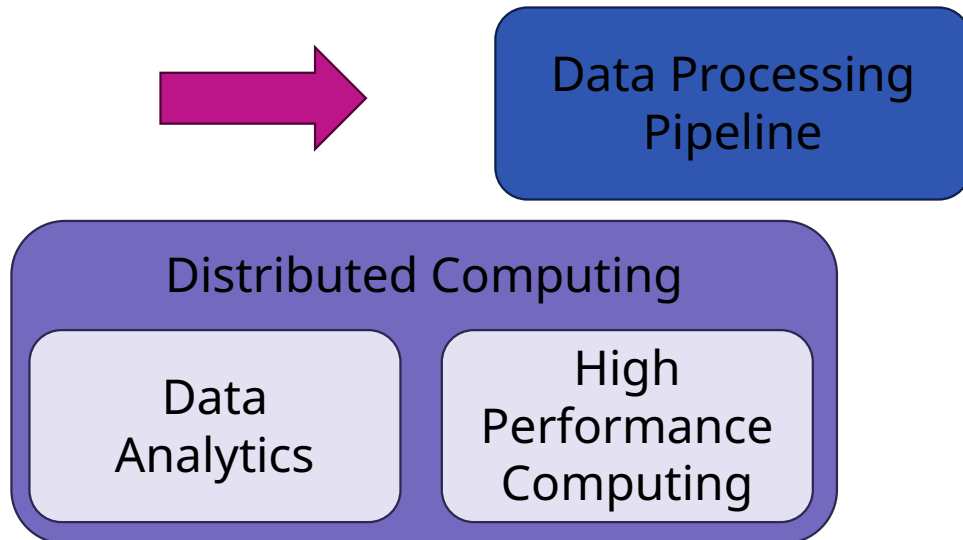
Lecture 10: Distributed Computing I

Lecture 11: Distributed Computing II

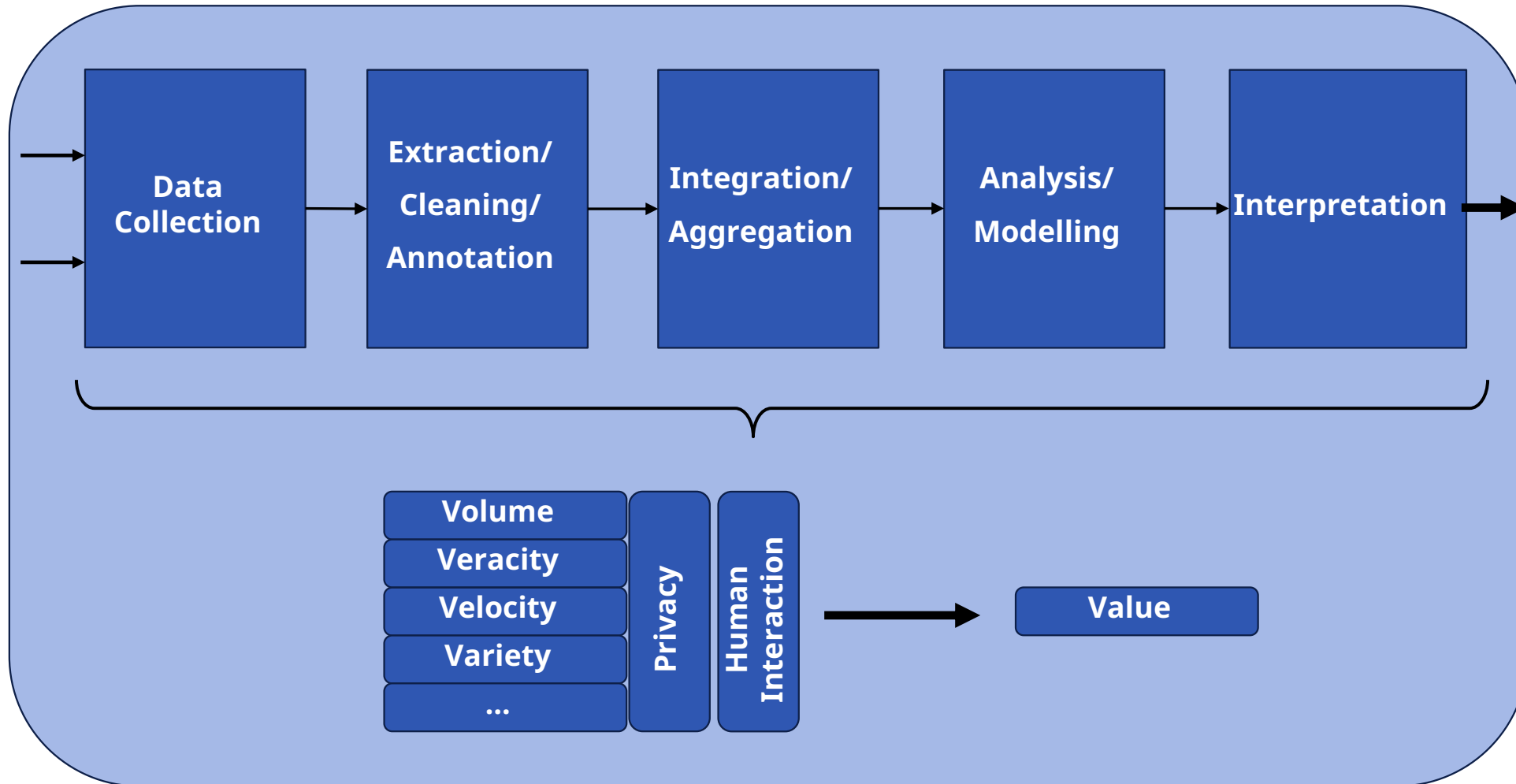
Lecture 12: Consequences

Data Analytics & Distributed Computing

Processing of large amount of data only possible due to **hardware evolution** and **new concepts** concerning **architecture, high performance computing, and processing**



Data Processing Pipeline



Challenges

Need for meaningful features derived from raw data

- Feature extraction

Analysis may require specific data type

- Data portability

Handle erroneous, inconsistent, or missing data

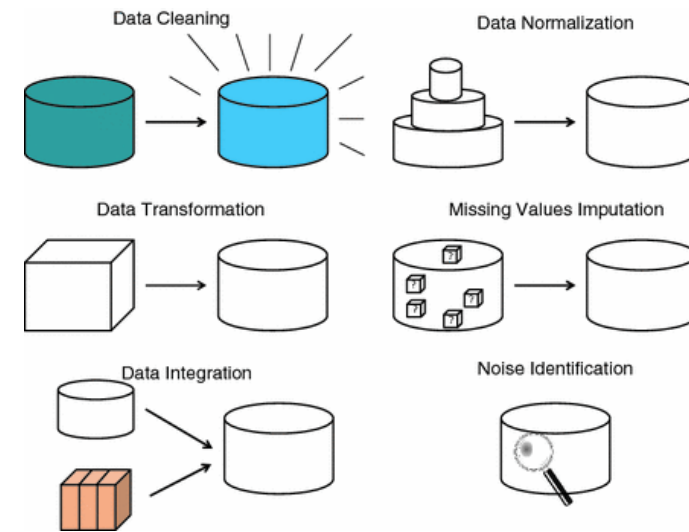
- Data cleaning

Data is obtained from multiple sources

- Data integration

Efficiency of Data Analysis

- Data reduction and transformation



Garcia, S. et al. Data Preprocessing in Data Mining, Springer, 2015



Technische
Universität
Dresden



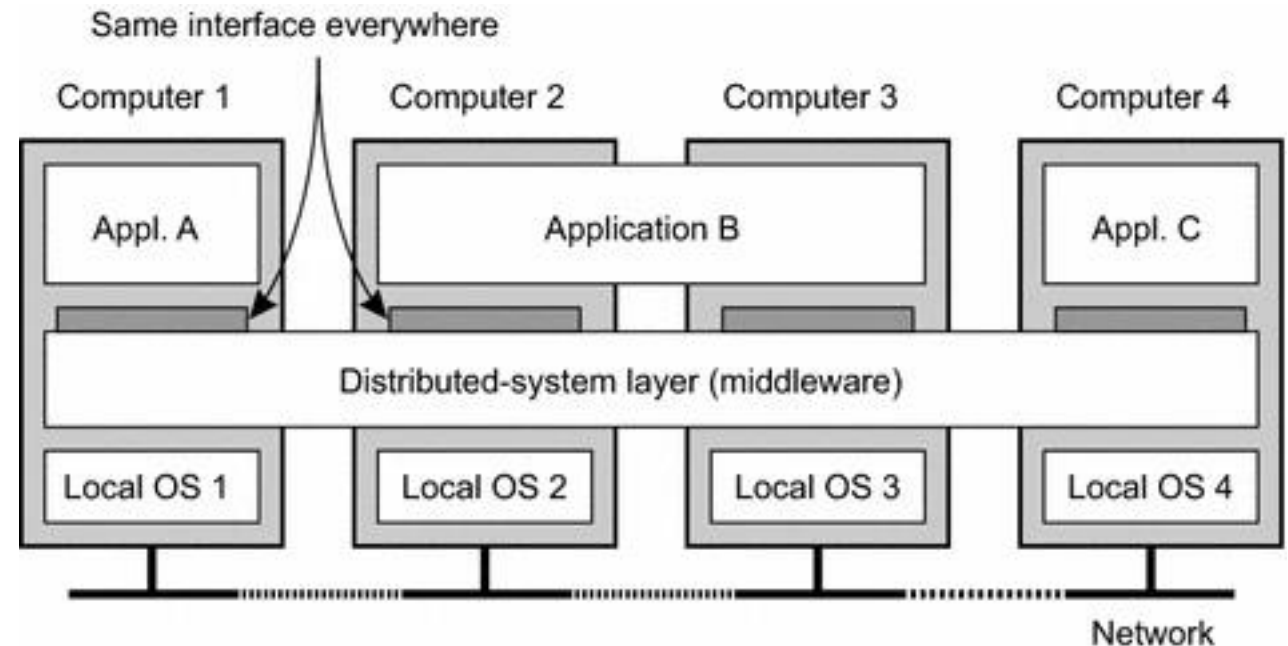
Distributed Computing

Definition

“A computer program that runs within a distributed system is called a distributed program, and distributed programming is the process of writing such programs. [1]

Distributing workload = Parallel processing (aka. "concurrent computing", "parallel computing", "distributed computing")

- message passing, e.g. MPI in HPC context for massive parallel applications
- Grid computing
- Cloud computing

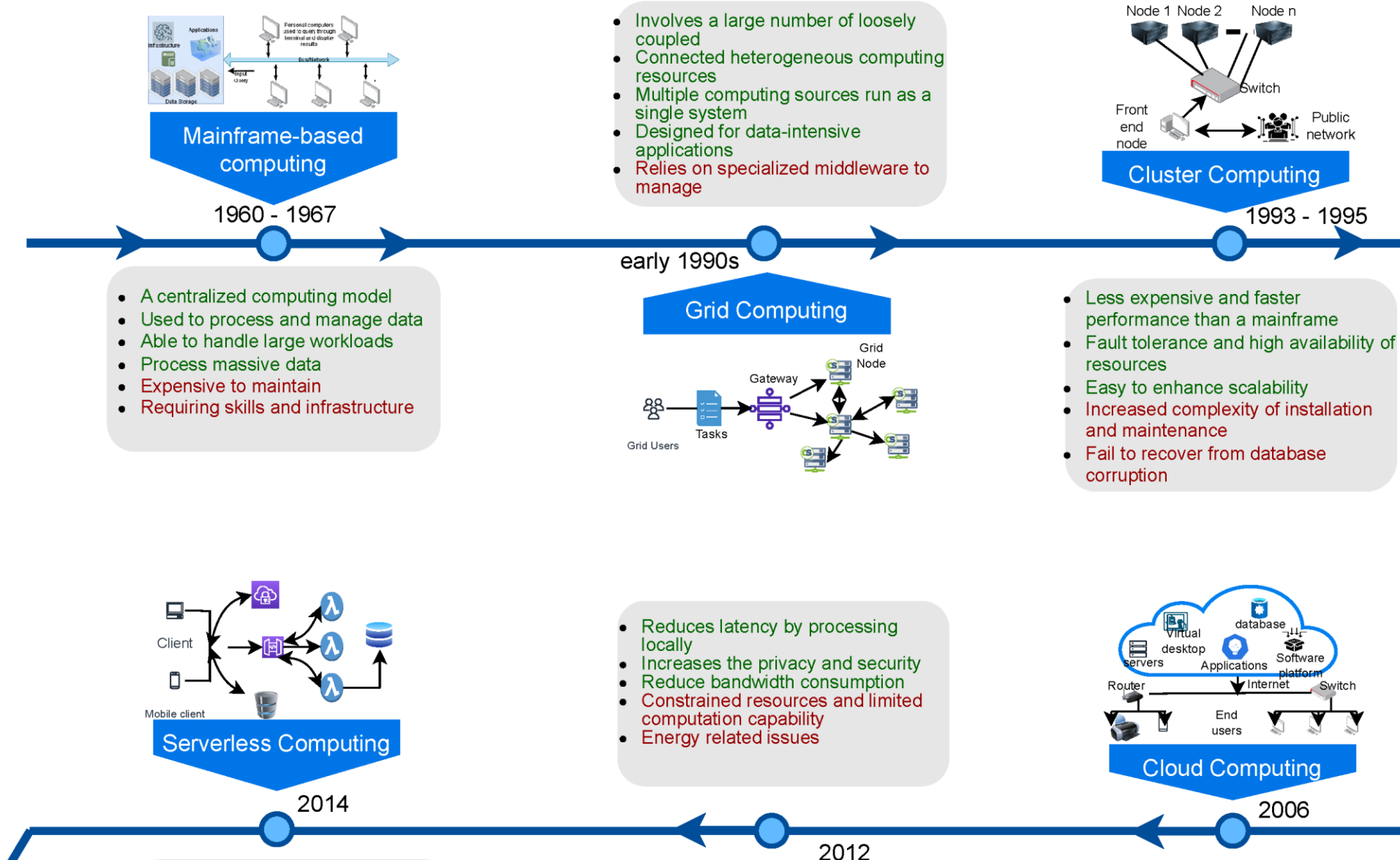


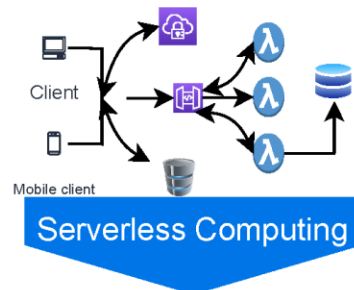
[1] https://en.wikipedia.org/wiki/Distributed_computing

[2] van Steen, M., Tanenbaum, A.S. A brief introduction to distributed systems. Computing 98, 967–1009 (2016).
<https://doi.org/10.1007/s00607-016-0508-7>

Development

Donta, P.K.; Murturi, I.; Casamayor Pujol, V.; Sedlak, B.; Dustdar, S. **Exploring the Potential of Distributed Computing Continuum Systems. Computers** 2023, 12, 198. <https://doi.org/10.3390/computers12100198>



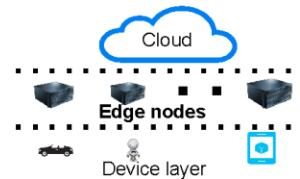


2014

- No Infrastructure to Maintain
- Easy scaling up or down of resources
- Reduced Software Development Time and Low latency
- Reduced Costs When Functions Aren't Running
- Privacy and Security Concerns
- Unstable Application Performance
- Limited control over resources

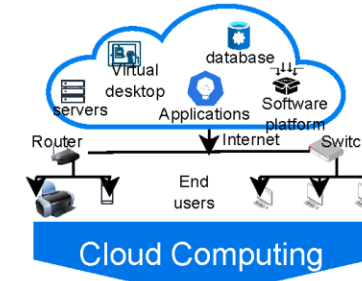
- Reduces latency by processing locally
- Increases the privacy and security
- Reduce bandwidth consumption
- Constrained resources and limited computation capability
- Energy related issues

Fog/Edge Computing



2012

- Efficient resource utilization
- Scalability by seamless integration
- Fault Tolerance and Real-Time Data Processing
- Facilitate collaboration and resource sharing
- Privacy and Security Concern
- Ensuring seamless interoperability and compatibility



2006

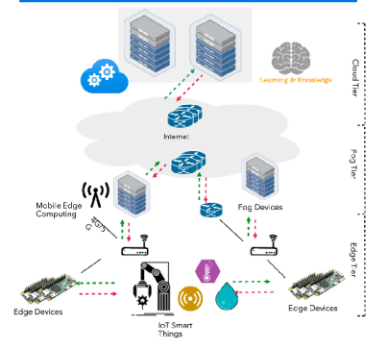
- Eliminating the need for on-premise hardware and software
- Easy scaling up or down of resources
- Enables easy collaboration among (remote) users
- Automatic Software Integration
- Privacy and Security Concerns
- High Latency and bandwidth needs
- Limited control over resources

Donta, P.K.; Murturi, I.; Casamayor Pujol, V.; Sedlak, B.; Dustdar, S. **Exploring the Potential of Distributed Computing Continuum Systems.** *Computers* 2023, 12, 198. <https://doi.org/10.3390/computers12100198>

- Efficient resource utilization
- Scalability by seamless integration
- Fault Tolerance and Real-Time Data Processing
- Facilitate collaboration and resource sharing
- Privacy and Security Concern
- Ensuring seamless interoperability and compatibility

2018

Distributed Computing Continuum Systems



Development

Lindsay, D., Gill, S.S., Smirnova, D. et al. **The evolution of distributed computing systems: from fundamental to new frontiers**. Computing 103, 1859–1878 (2021). <https://doi.org/10.1007/s00607-020-00900-y>

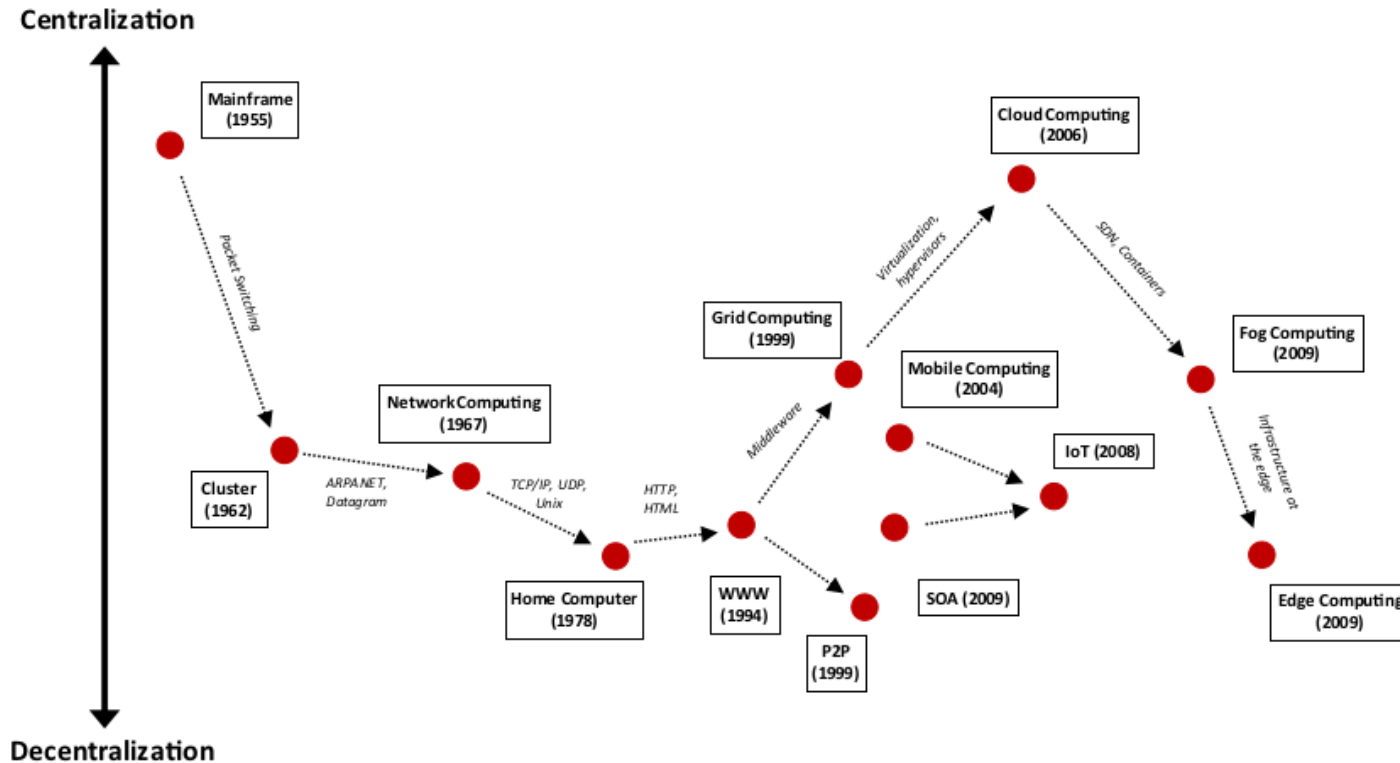


Fig. 1 Depiction of distributed system paradigm evolution

Key challenges

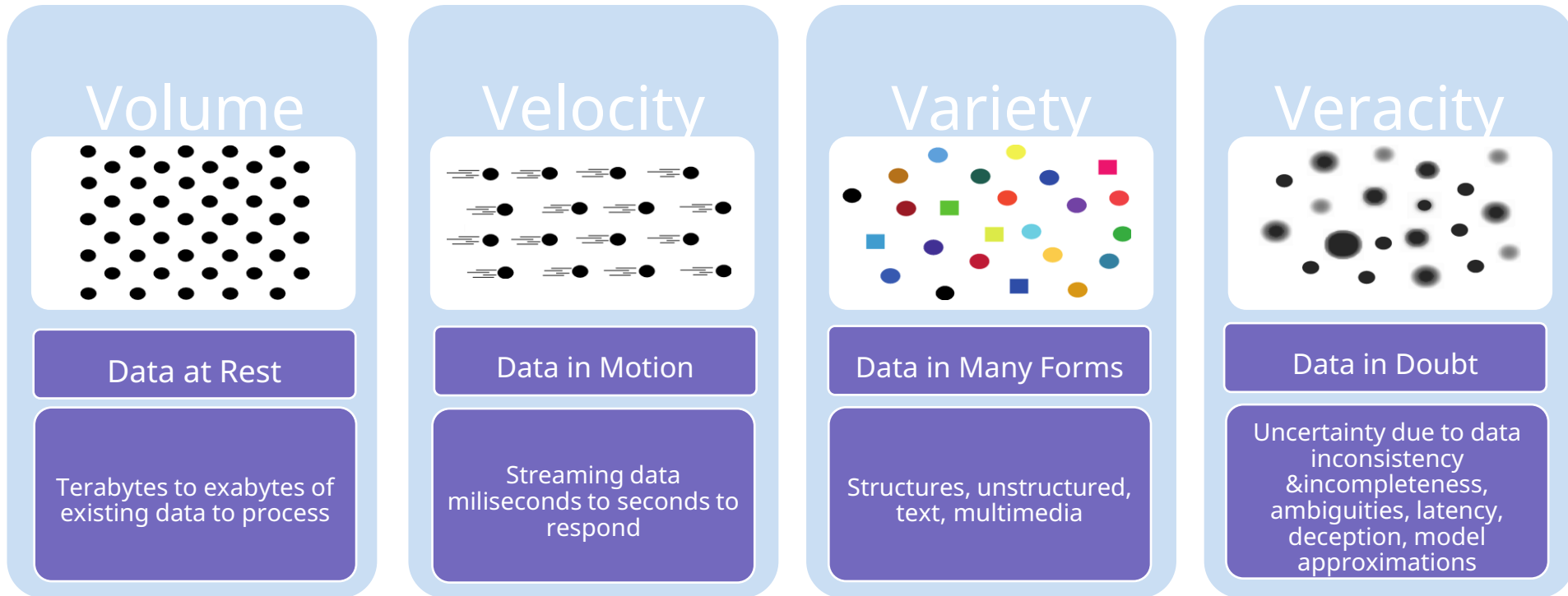
Process (analyze) massive amounts of data

- Scalability
- Data management
- (Data-) security and Quality

Management of infrastructure

- (heterogeneous) instances
- Service orientation
- Fault tolerance
- Communication
- Security

Driving force for distributed computing: Big Data (Definitions)

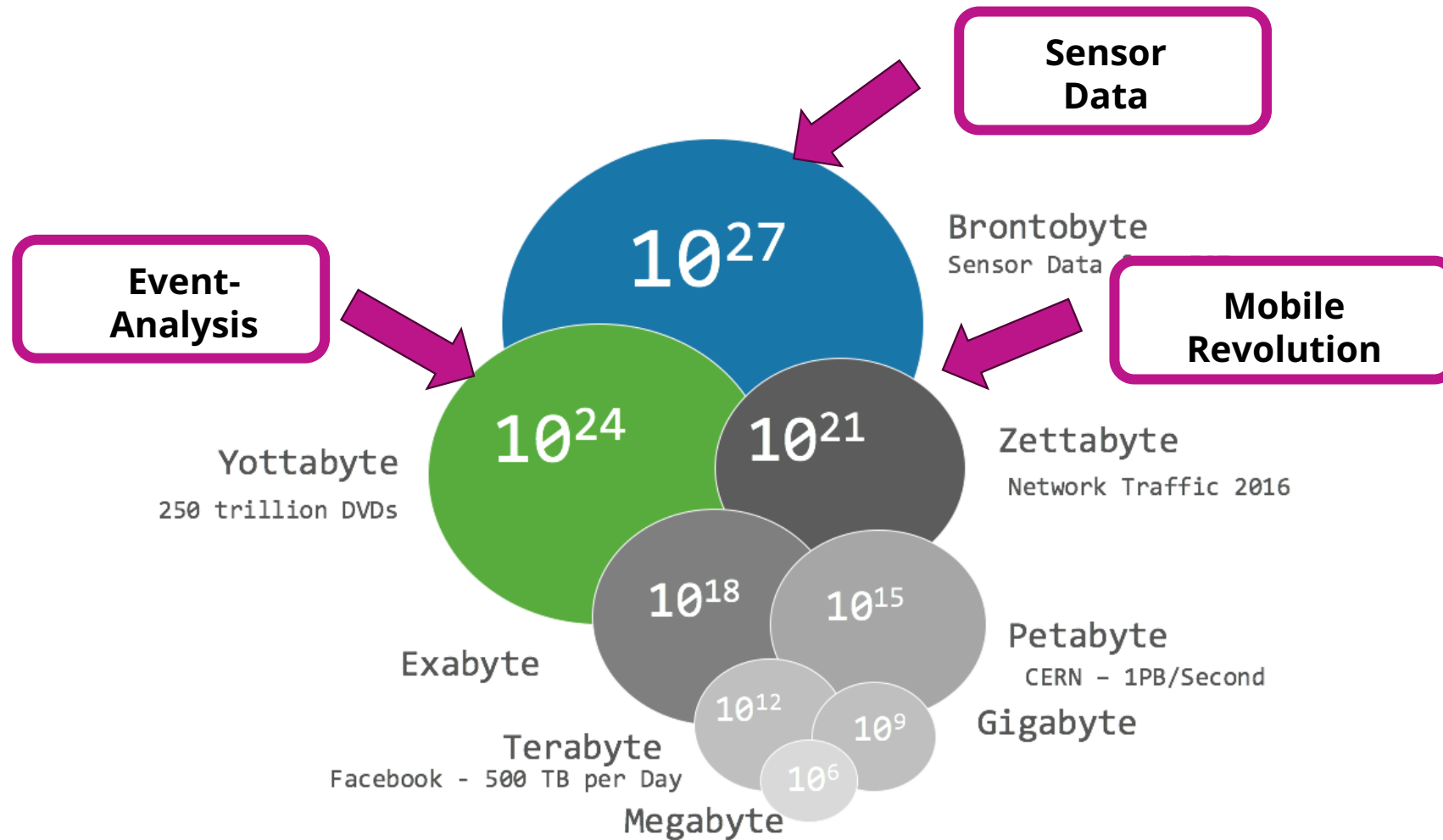


More important:

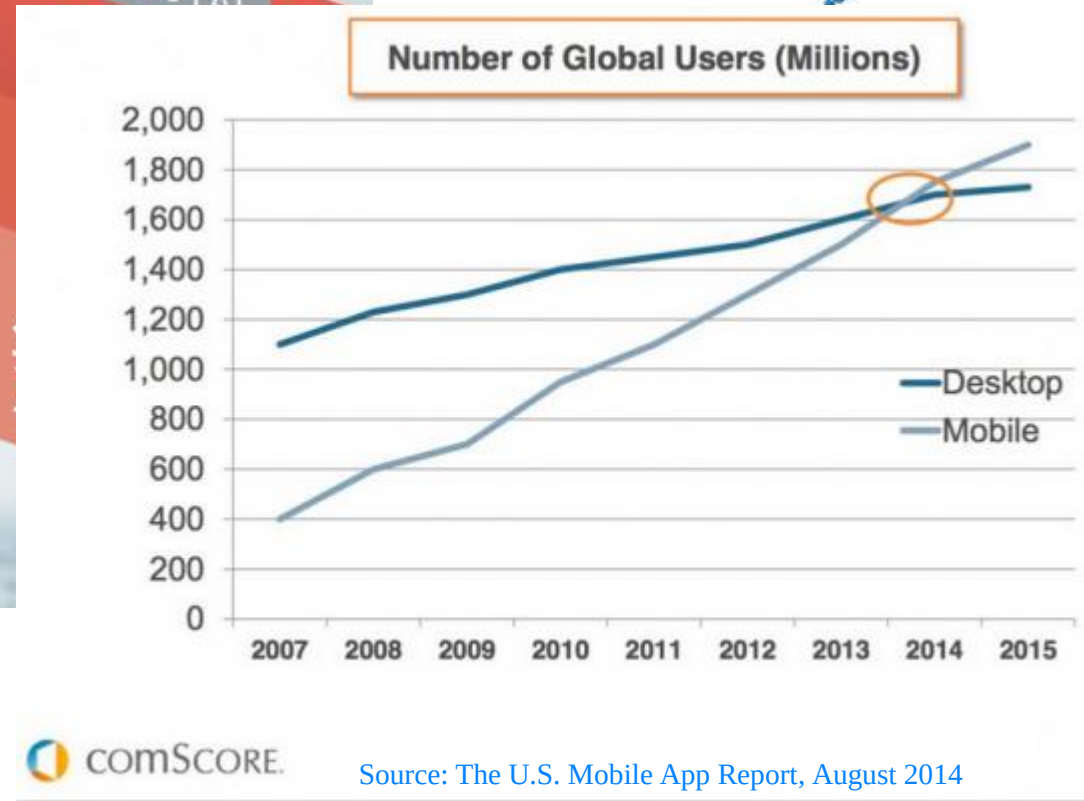
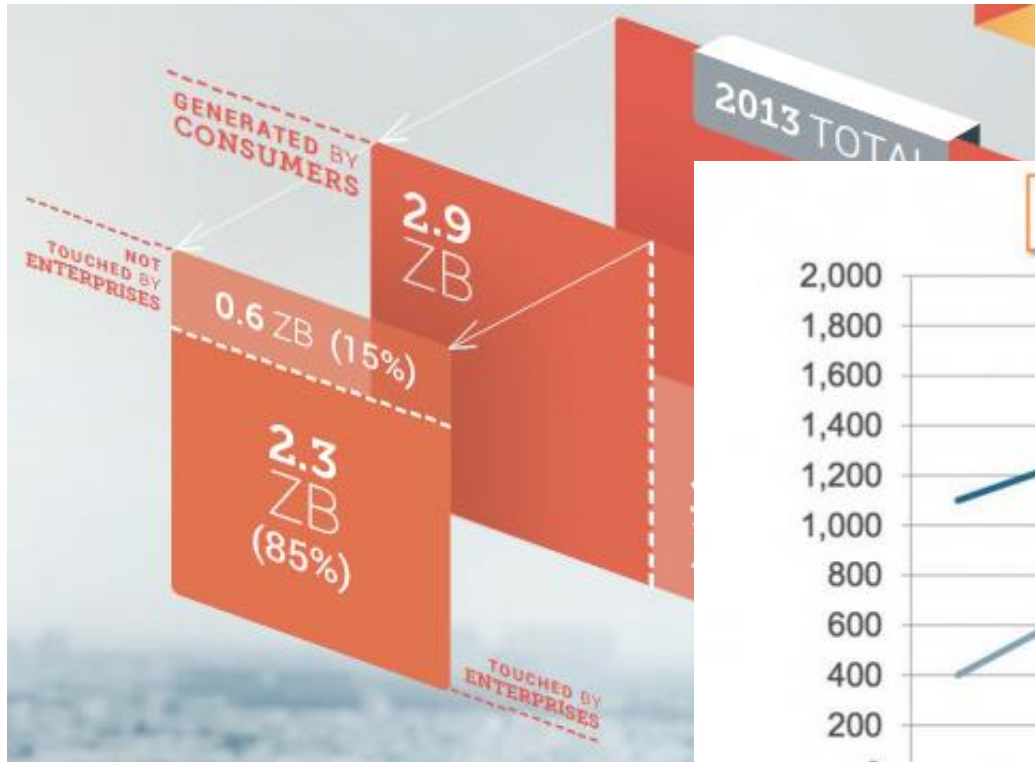


extract new content from database

Where is Big Data coming from?



Where is Big Data coming from?



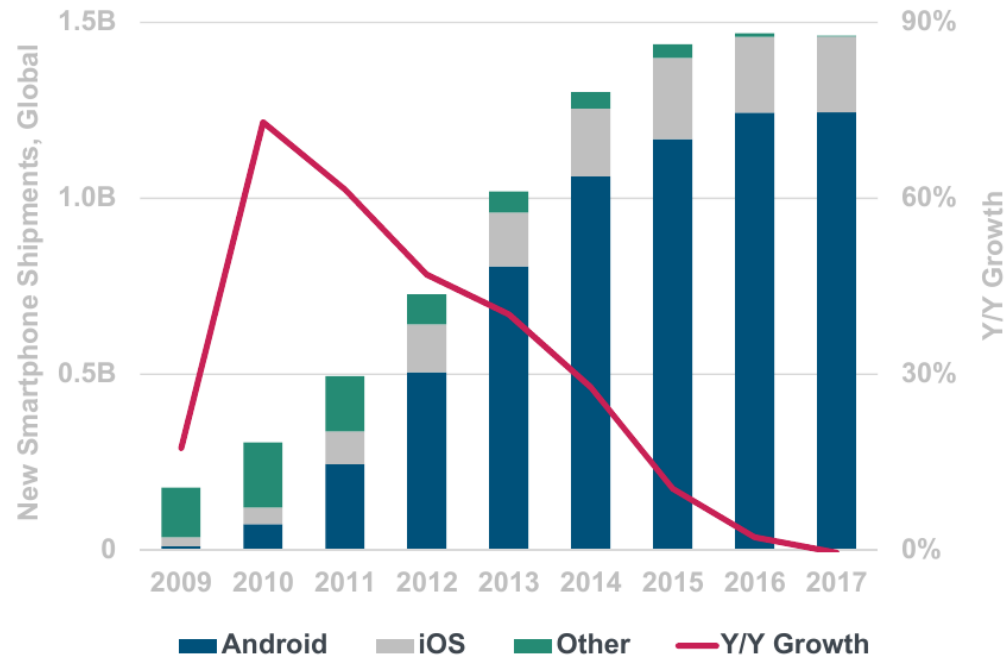
Sources of data – interaction and availability of technologies and access to internet

KLEINER PERKINS
2018
INTERNET TRENDS

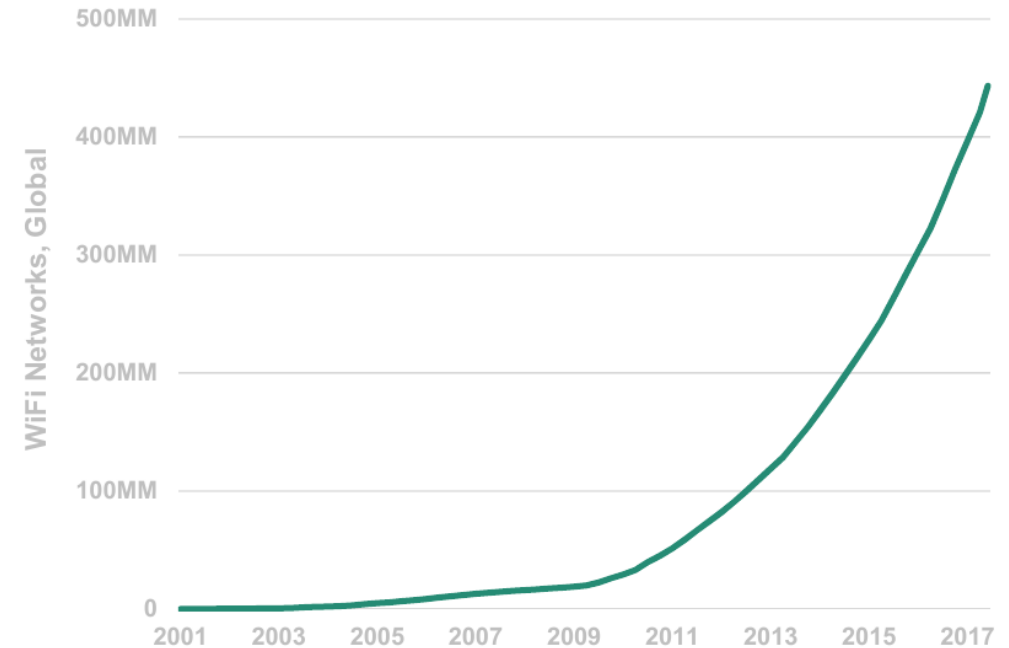
Source: Katy Huberty @ Morgan Stanley (3/18), IDC.

<https://www.kleinerperkins.com/perspectives/internet-trends-report-2018>

New Smartphone Unit Shipments vs. Y/Y Growth



WiFi Networks



What is the motivation to collect and analyze huge 'piles' of data?

<https://www.businessnewsdaily.com/10625-businesses-collecting-data.html>

<https://www.pointillist.com/blog/customer-behavior-data/>

Basic goals:

Derive **knowledge** out of data / information

Use 'analytics' to find **correlations/causality, systematic behavior**: use machine learning or some sorts of AI...

Driving force in businesses:

- Improving customer experience: Optimization of services, direct feedback, recommendations
- Refining marketing strategy: collect and quantify response of marketing – “understand” your customer
- Turning data into cash-flow: secondary data markets; share of behavioral data amongst companies
- Using data to secure data: secondary data e.g. in two-factor authorization (voice, biometric data)
- Etc. ...

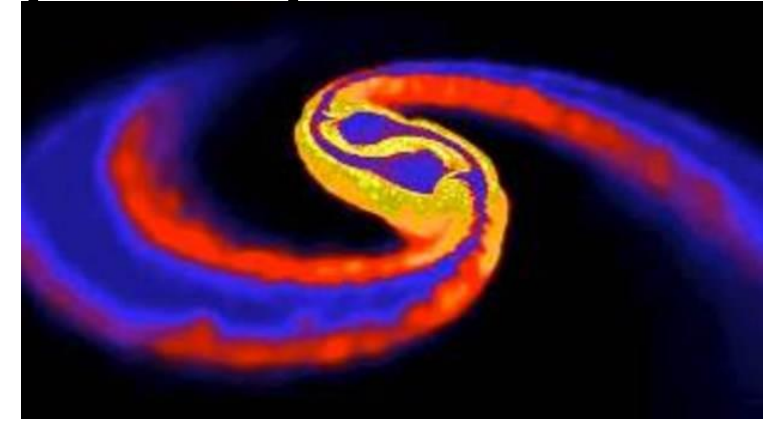
Why is it different from scientific computing?

Long history in scientific studies:

Theory based hypothesis: Example relativistic theory

➡ prediction of gravitational waves

Data driven approaches: no theory available (so far?)



Merging neutron stars. (Image: Price & Rosswog)
<https://www.ligo.caltech.edu/page/learn-more>

- Socio-economic system: is there a theory which can be used to formulate hypotheses?
- Statistics: derive global properties, probabilistic distributions
- General behavior of distributions of data properties
- Machine learning: approximate functional dependencies in data based on hypothesis (supervised) or no hypothesis (unsupervised), purely data driven

New trend in DA: Fusion of simulated/syntetic and measurement data

- ML: “Physics informed” networks (use or real-world constraints in state of model)
- Digital Twin: digital copy of a real-world object/system combining data and model(s)

History & evolution of big data – hardware aspects

2000

Hardware

- *Disk space* **cheap** (primary storage solutions)
- *Network* was **costly**
- *RAM* was very **expensive**
- *Single core* machines were dominant

Software

- Object orientation and optimization for **single core**
- SQL as primary analysis language
- some specific frameworks (Mathlab)

Now (~201a)

Hardware

- *RAM/Flash* got **cheaper** and **faster** – tend to become primary storage, disk as fall-back
- Network is **faster** / Virtualization
- **Multi core** machines are dominating, different architectures

Software

- More functional programming and **frameworks**
- **Multicore-programming** and distribution
- **NoSQL alternatives**

History & evolution of big data – software aspects

200x: First generation of data processing frameworks (Big Data)

Purpose/Background

- ONLY few companies had real big data need
- Batch processing was still dominant way to distribute workloads (reporting++)
- Primarily **volume** was biggest concern (not 5 V's)
- Mostly used for **search/basic behavior** analysis (logging data)

Hadoop-implementation offered simple programming approach

- Batch orientated
- Underlying HDFS for data distribution

History & evolution of big data – software aspects

201x: Second generation of data processing frameworks (Big Data)

Purpose/Background

- Most companies have need to use big data technologies
- **Velocity** now dominating, also “Value”
- Diverse use cases
 - **Real time processing** of data
 - **Learning approaches, iterations, interactions**

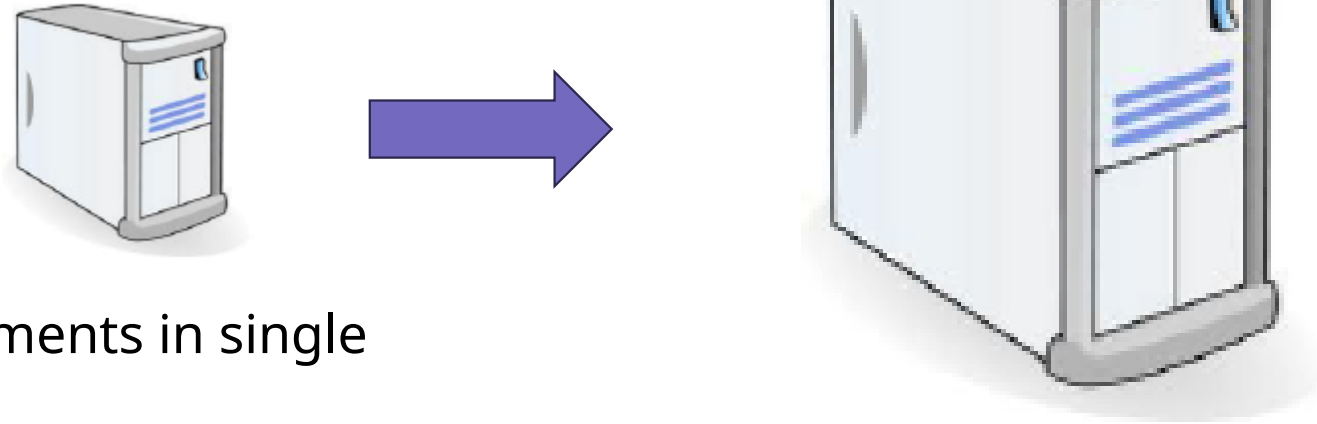
Hadoop-approach not flexible enough any more

- Need for **in-memory processing** (handle iterations efficiently)
- Very **flexible hardware** options (Shared-Nothing vs. HPC)

Vertical Scaling – massively parallel applications (HPC)

Scale-up

Usually within a system (e.g. scale parallel application)



Use more computing elements in single application

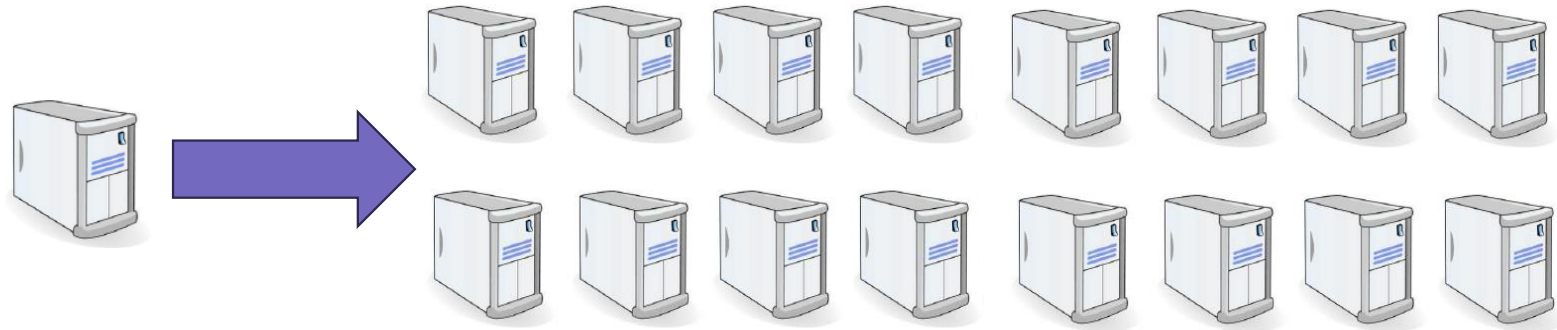
Often communication intensive (iterative approaches)

Ideal case: highly parallel simulations

Horizontal Scaling – “trivial parallelism”

Scale-out

Opposite to scale-up



Add further instances of the same kind

No special assumptions concerning hardware dependencies

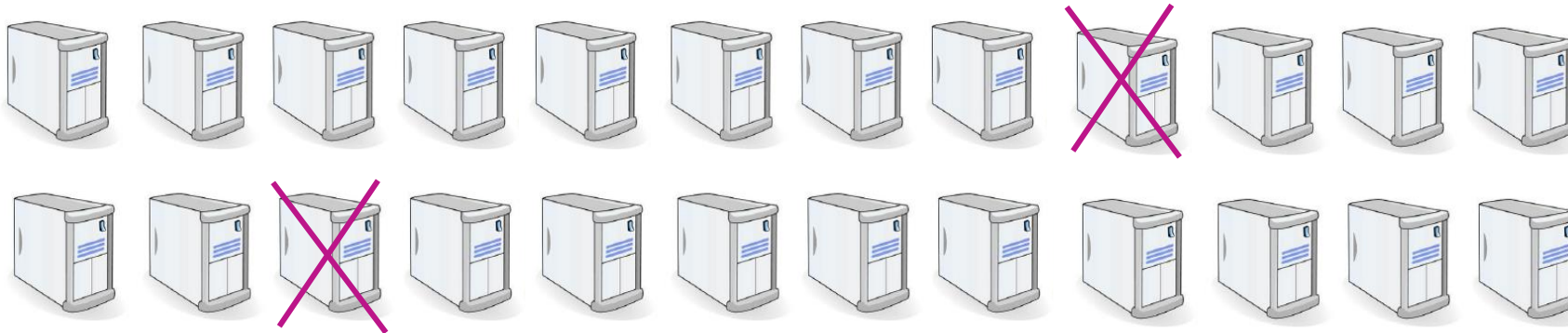
Ideal case: distribute computing to data, not vice versa,
“separate” partitioned workloads

Data Processing and Analytics

Why not just taking 'low-cost' hardware like standard server components?

Scale out paradigm

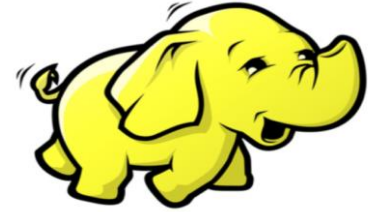
Many machines (hundreds, thousands)



As opposed to scale-up, where one very powerful (single) server is used

Do not rely on each instance!

MapReduce Paradigm



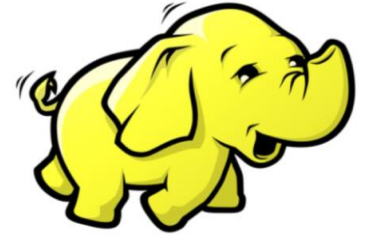
How to process and store data on this infrastructure?

Scale out paradigm:

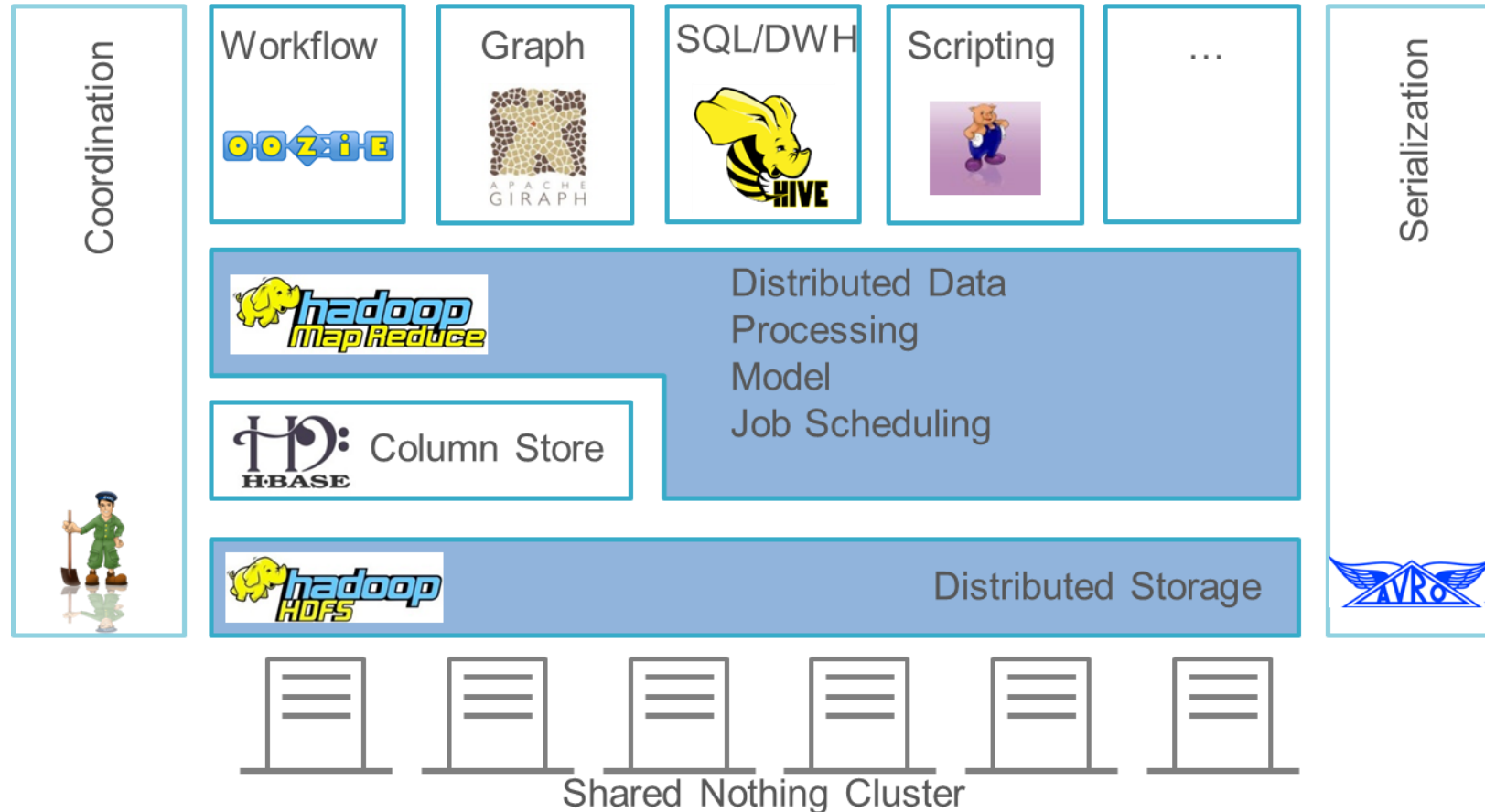
- The Google File System (**Sanjay Ghemawat**, H. Gobioff, S. Leung. SOSP 2003)
- MapReduce: Simplified Data Processing on Large Clusters (**Jeff Dean, S. Ghemawat**. OSDI 2004)
- Hadoop is **open-source** software framework
- Supports data-intensive **distributed** applications and clones the Google's MapReduce
- Designed to process very **large amounts** of unstructured and complex **data**
- Designed to run on a **large** number of machines



Hadoop 1.0 Ecosystem

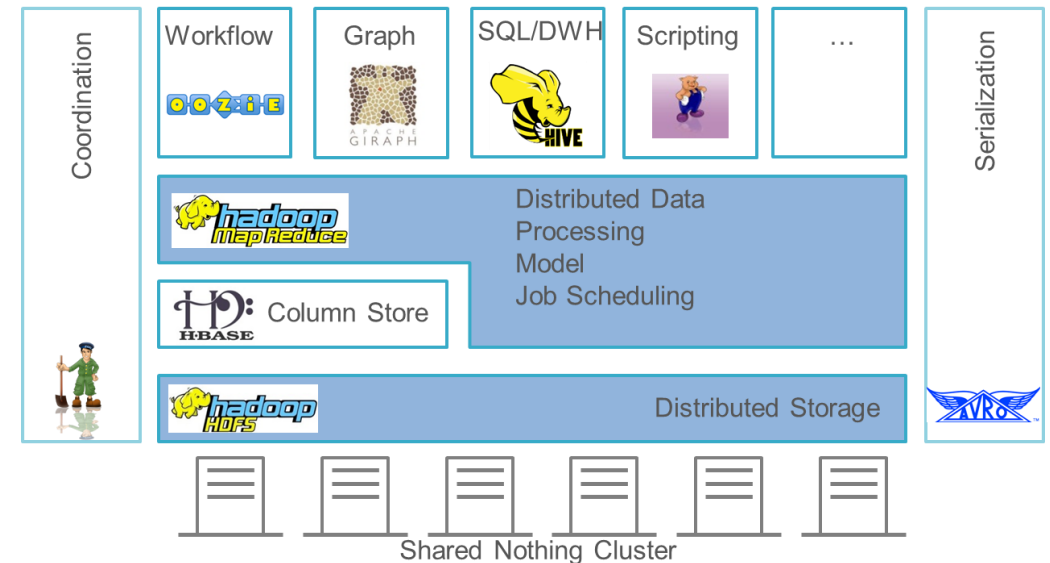
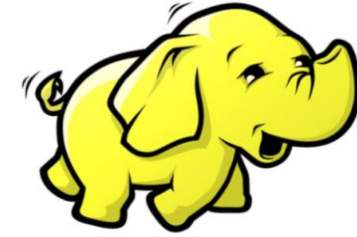


Not just simple functions – towards interoperable frameworks



Assumptions and Goals

- Hardware Failure
 - Hundreds to thousands of commodity servers per cluster
 - Failure is the norm rather than the exception
- Streaming Data Access
 - Applications need streaming access to the data
 - Batch processing rather than interactive use
- Large Data Sets
 - Tens of millions of files each with gigabytes to terabytes in size
- Moving computation to data
 - Computation „near“ the data it operates on
 - Knowledge about data location is needed
- Portability
 - Run on heterogeneous hardware and software platforms



HDFS

HaDooP File System

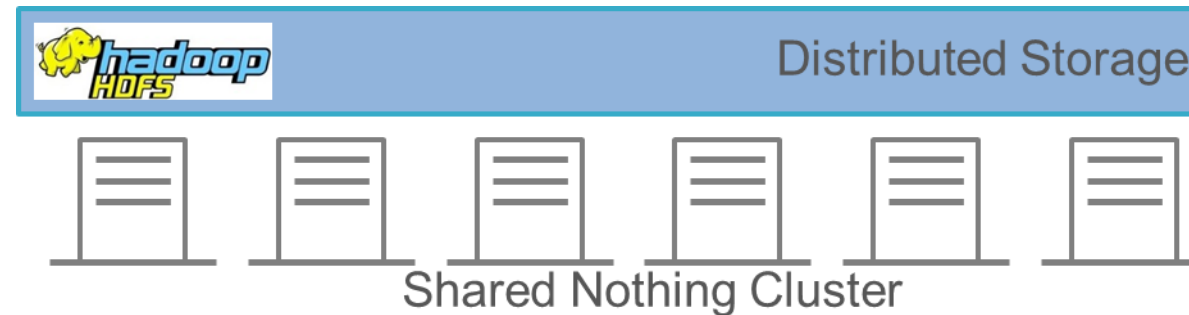
File System Component of Hadoop

Open Source under Apache License, Version 2.0

Initially created at Yahoo!

Written entirely in Java

Typically runs on GNU/Linux operating systems



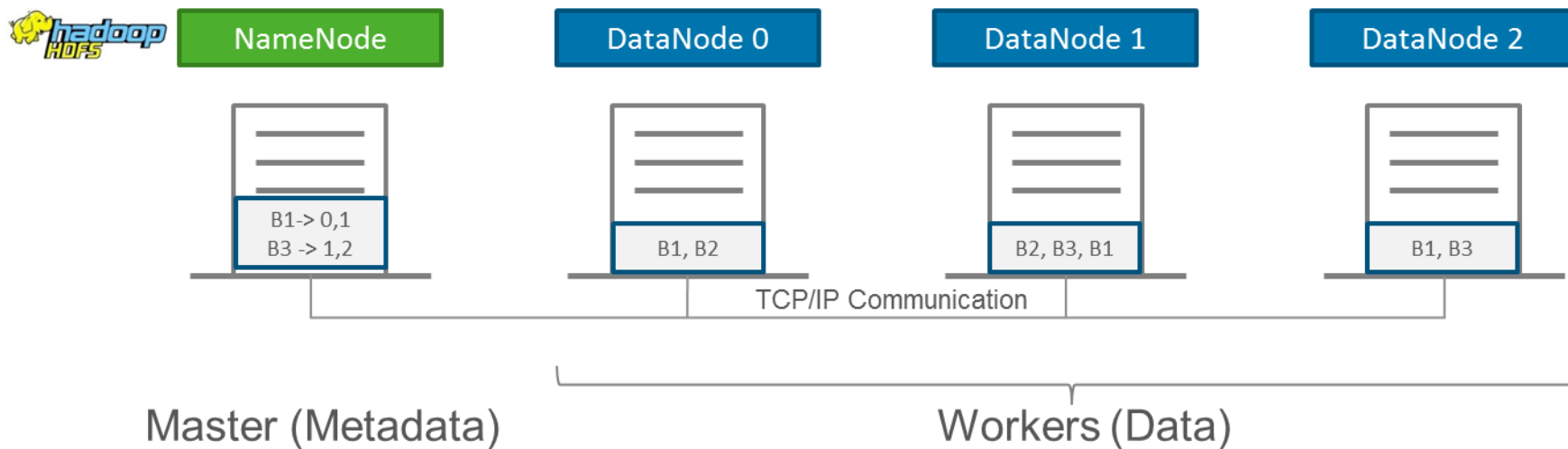
HDFS

HDFS files are broken into block-sized chunks

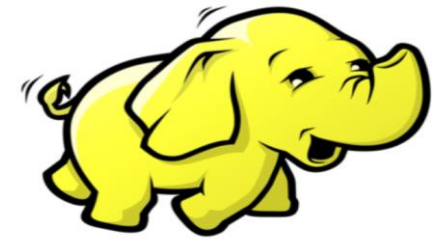
Blocks are basic unit for replication: no RAID but real duplicates

Blocks is input for a processing task

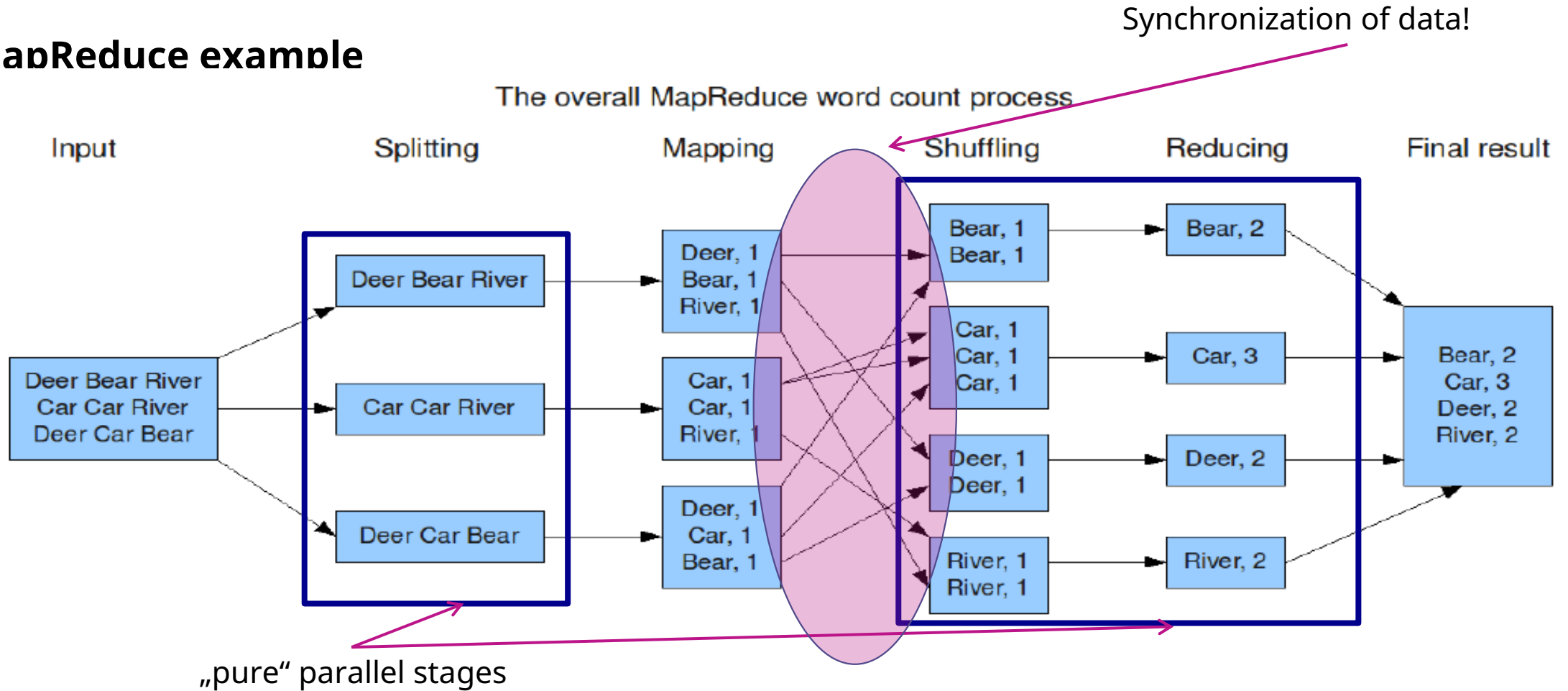
MyBigFile is usually split into chunks of data: B1, B2, B3



Hadoop 1.0 Ecosystem



MapReduce example



MapReduce criticism (from the DBMS perspective - 2008)

Criticism from DB research perspective:

Programming paradigm for large-scale data intensive applications – step backwards

Missing core DBMS features

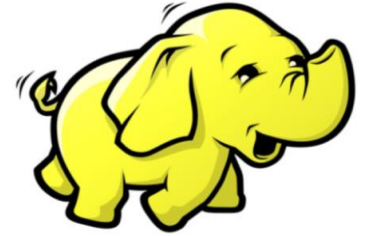
Incompatible with many tools in DBMS-Worlds

Missing features:

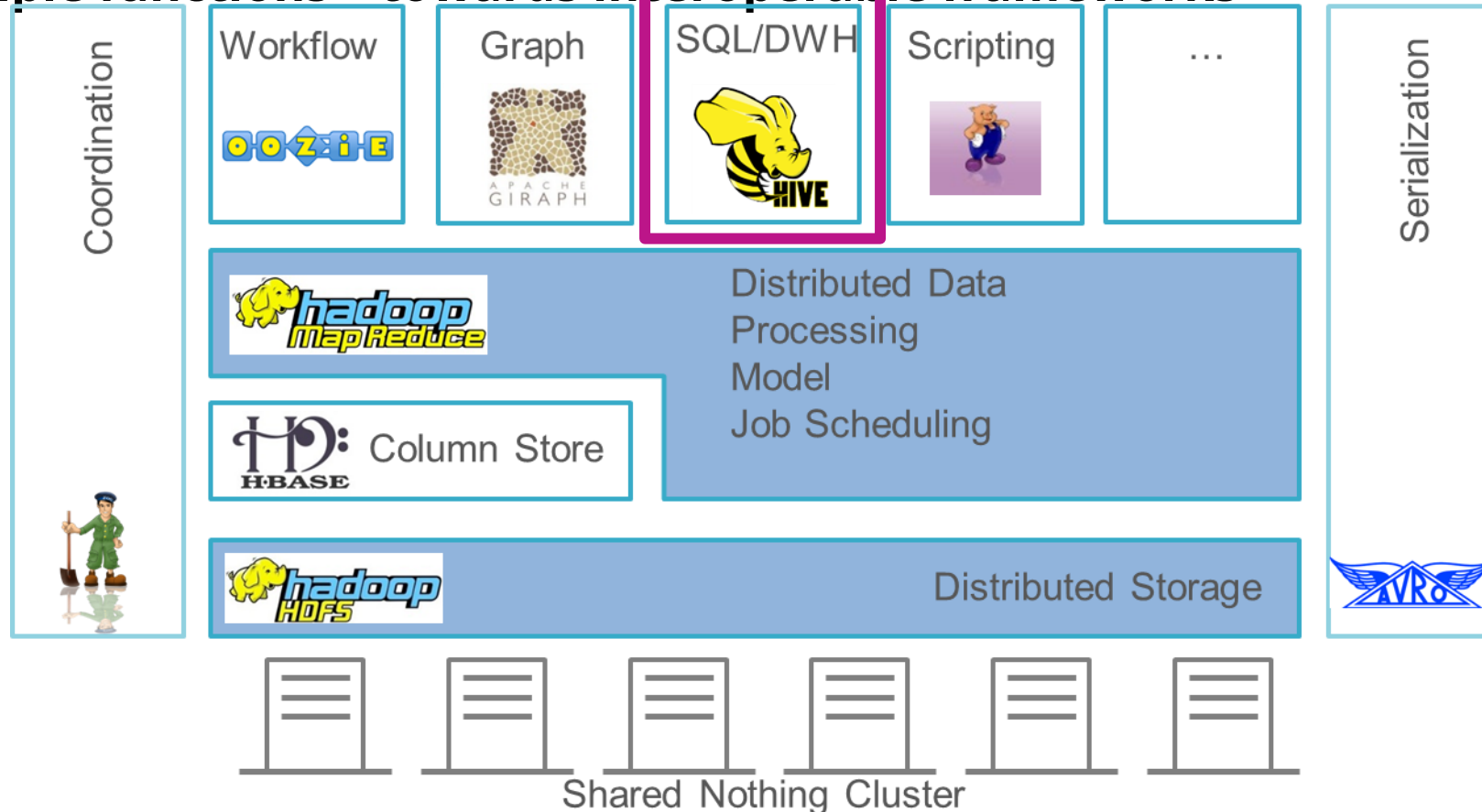
- Indexing – simple MapReduce has no index, only “brute force”
- Updates -- to change the data in the data base
- Transactions -- parallel update & recovery from failures during update
- Integrity constraints and references
- Views -- schema change without rewriting the application program

https://homes.cs.washington.edu/~billhowe/mapreduce_a_major_step_backwards.html

Hadoop 1.0 Ecosystem



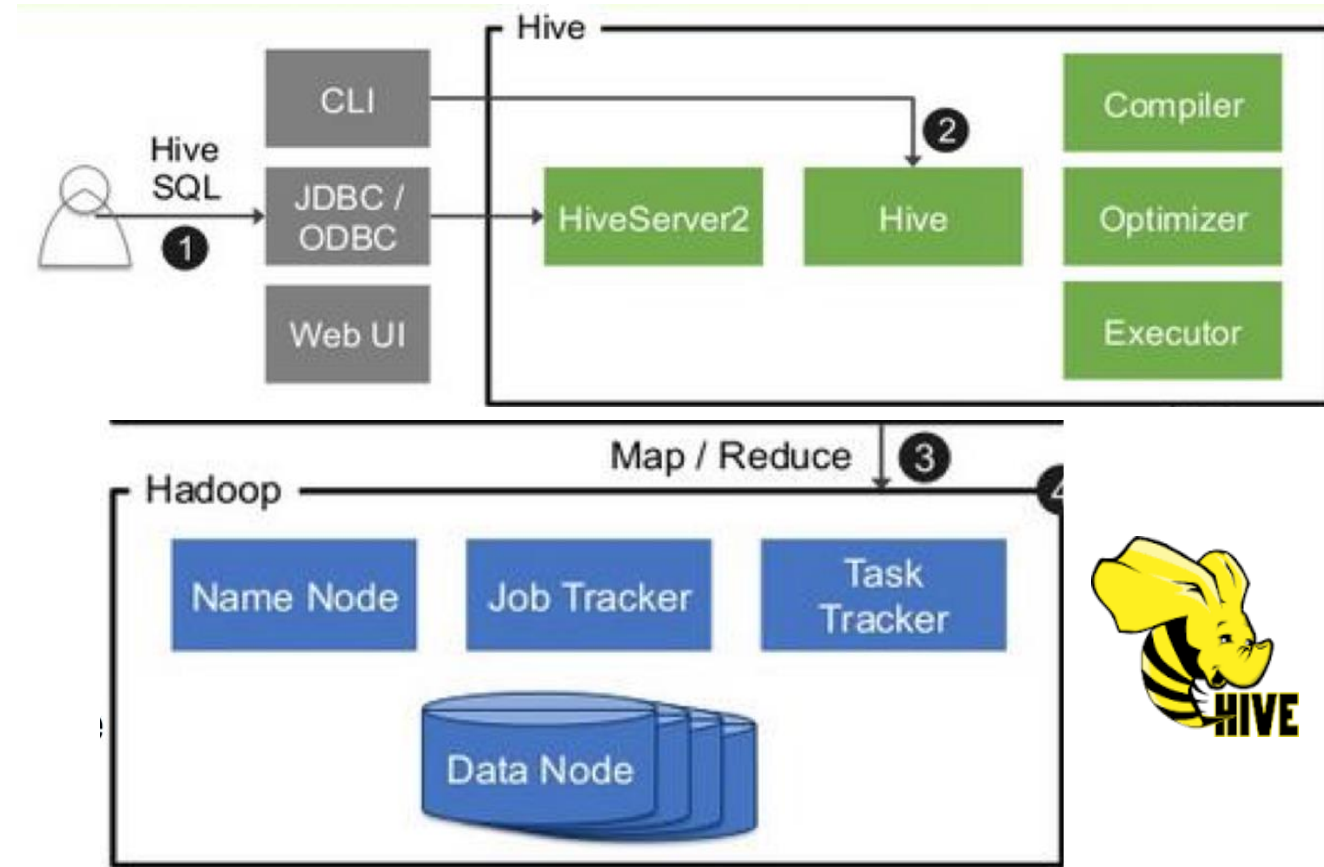
Not just simple functions – towards interoperable frameworks



Apache HIVE – the data warehouse for MapReduce

- Strongly influenced by data warehouse concepts
- Hive = MapReduce + SQL
 - SQL simple to use
 - Keep MapReduce scalability and fault tolerance mechanism
- HiveQL = SQL-like query language
 - No need to implement queries in the low-level Java API
 - Extensible with MapReduce-Scripts

1. User issues SQL query
2. Hive parses and plans query
3. Query converted to MapReduce
4. MapReduce tasks run by Hadoop



Hive query usage

Hive is still widely used as extension in the Hadoop ecosystem (first released 2010)

Initially developed by Facebook

Still used by other companies (e.g. Netflix and FINRA (Financial Industry Regulatory Authority))

Amazon developed on version of Hive for its Amazon Web Services to be included into Amazon Elastic MapReduce

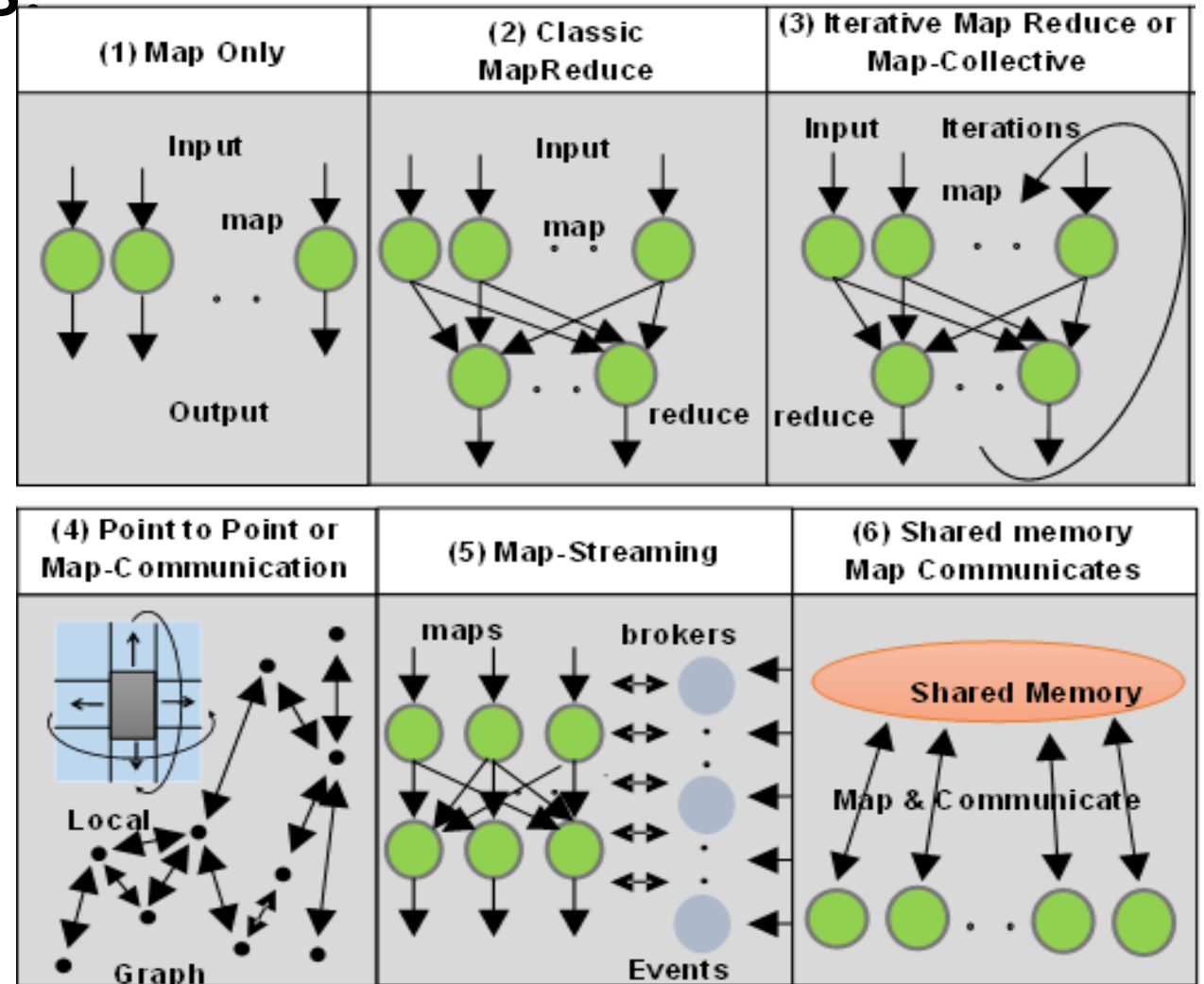
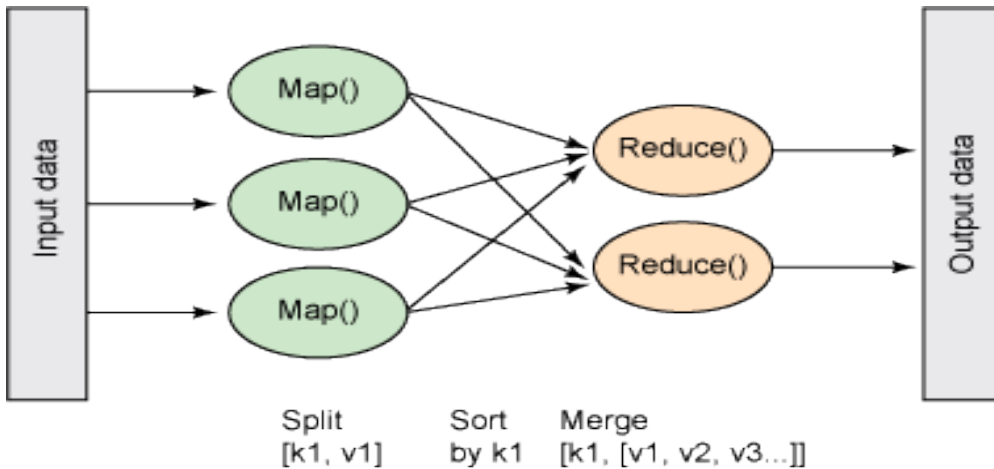
```
DROP TABLE IF EXISTS docs;  
CREATE TABLE docs (line STRING);  
LOAD DATA INPATH 'input_file' OVERWRITE INTO  
TABLE docs;  
  
CREATE TABLE word_counts AS  
SELECT word, count(1) AS count FROM  
  (SELECT explode(split(line, '\s')) AS  
    word FROM docs) temp  
  
GROUP BY word  
ORDER BY word;
```



MapReduce – Limitations!

Widely adopted processing model, but:

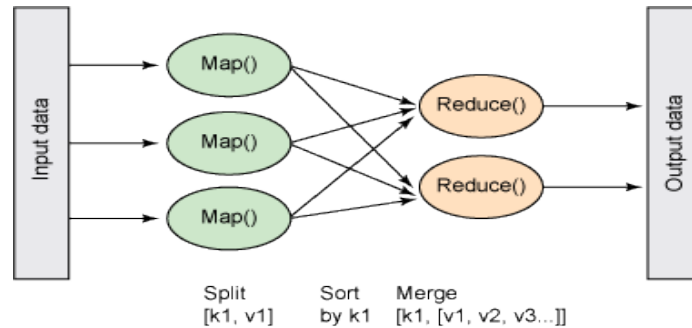
- Non-Interactive
- Simple programming model but complex to transform tasks in map reduce model
- No iterations



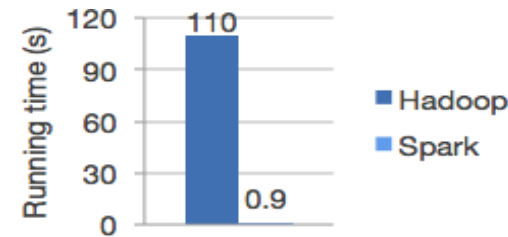
[Geoffrey Fox: Lecture: Building a Library at the Nexus of High Performance Computing and Big Data, Indiana University]

Data-driven analytics – types of analytics scenarios

Batch-processing



In-memory computing



Stream-processing



Batch oriented:

- Ask a specific question to bunch/collection of data

In-Memory processing:

- Load all data into memory and perform multiple operations on it (avoid staging data (out) if possible)

Stream Processing:

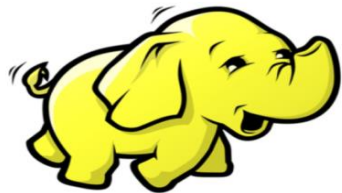
- Analyze continuously changing data, e.g. over time or in its specific content or interpretation, where historical background is not important

Processing challenges - data intensive applications (Big Data)

- Not „just“ Scalability – further challenges
- **Data Acquisition:** “capturing” data, streaming or persistent storage of data
- **Aligning / Integration** of data from different sources (e.g., resolving duplicates)
- **Transformation:** bring data into a form of suitable format for further processing or analysis
- **Modelling:** hypothesis testing or prediction, usually mathematically or statistical
- **Understanding** the output — visualizing and sharing results
- Techniques:
 - Massive Parallelism
 - Huge Data Volumes Storage
 - Data Distribution
 - High-Speed Networks
 - High-Performance Computing
 - Task and Thread Management
 - Data Mining and Analytics
 - Data Retrieval
 - Machine Learning
 - Data Visualization

Big Data 2.0 processing platforms

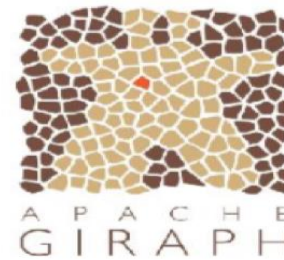
General Purpose



Big SQL



Big Graph



Big Stream



Apache Projects (relevant for Big Data)

Big data (36):

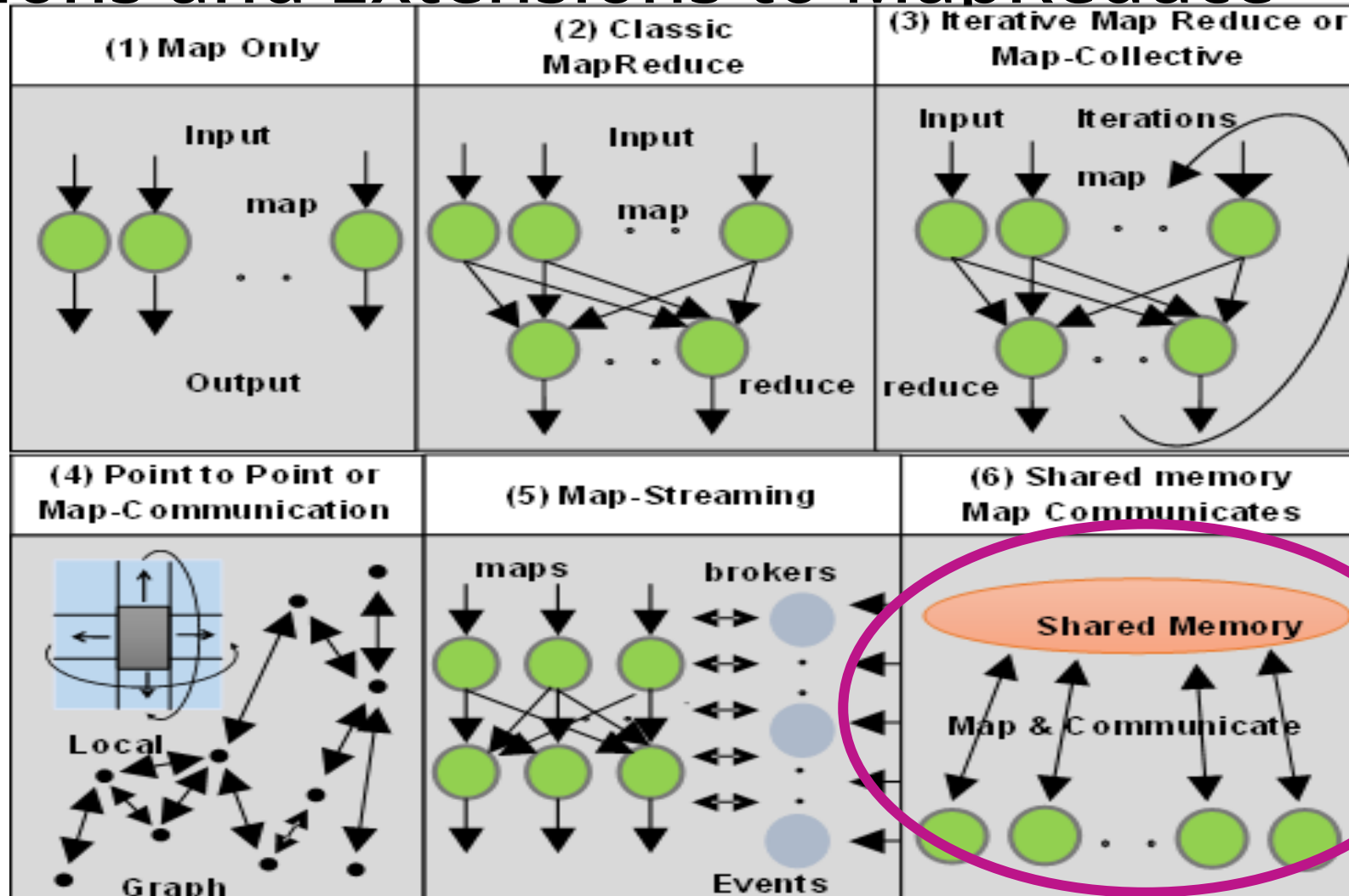
- Airavata, Ambari, Apex, Avro, Bigtop, BookKeeper, Calcite, CouchDB, Crunch, DataFu (Incubating), DirectMemory (in the Attic), Drill, Falcon, **Flink**, Flume, Giraph, Hama, Helix, Ignite, Kafka, Knox, MetaModel, Oozie, ORC, Parquet, Phoenix, Quarks (Incubating), REEF, Samza, **Spark**, Sqoop, Storm, Tajo, Tez, VXQuery, Zeppelin

Database (25):

- Accumulo; Cassandra, Cayenne, Cocoon, CouchDB, Curator, Derby, Empire-db, Forrest, Gora, **Hadoop**, Hbase, Hive, Jackrabbit, **Lucene Core**, **Lucene.Net**, Lucy, MetaModel, OFBiz, OpenJPA, ORC, Phoenix, Pig, Torque, ZooKeeper

Many more: <https://projects.apache.org/projects.html?category>

Limitations and Extensions to MapReduce



In-Memory
Processing

[Geoffrey Fox: Lecture: Building a Library at the Nexus of High Performance Computing and Big Data, Indiana University]

Summary

Distributed Computing: connection of hardware / software challenges and new concept to provide computing concepts

Introduction of data-centric analytics approach was driving force of distributed computing approaches, such as Grid and Cloud Computing

Definition of simple programming model via MapReduce provided access to large-scale distributed data sets

Driven by technological advancements (cheaper mass storage beneficial for MapReduce and its simple analytics approach)

“Big Data” approach were further developed to overcome limitations, namely IO-bottleneck and to introduce further analytics functionalities (not just mapping and reducing, but complex analytics chains)